CIPHERSHARE — SECURE CLOUD FILE SHARING

Technical Project Report

Group - 22
Group Members:
Sarvjeet Swanshi - S20230010216
Gourav Khakse - S20230010093
Saurav kumar roy - S202300010218

1. Introduction

Cloud storage providers such as Amazon Web Services (AWS S3), Microsoft Azure, and Google Cloud offer Service Level Agreements (SLAs) guaranteeing high availability (e.g., 99.9%), but provide zero contractual guarantees regarding data confidentiality, integrity, or access control. Under traditional server-side encryption architectures, the cloud provider holds the decryption keys, leaving files vulnerable to insider threats, hypervisor breaches, and subpoena access.

CipherShare solves this foundational vulnerability by implementing **Client-Side Hybrid Cryptography**. Grounded in the theoretical framework of **Popa et al. (CloudProof, USENIX ATC '11)**, CipherShare enforces end-to-end zero-knowledge secrecy: bulk file payloads are encrypted client-side in browser volatile memory using **AES-256-GCM**, while content keys are encapsulated using **RSA-OAEP 3072-bit** asymmetric cryptography. The cloud acts strictly as an untrusted blind storage repository with zero plaintext or private key knowledge.

1.1 Client-Side Encryption vs. Traditional Cloud Storage

Storage Architecture	Plaintext Visibility	Content Key Custody	Tamper Detection	Access Isolation
Traditional Cloud (S3 / Azure)	Cloud server sees plaintext in RAM	Cloud provider KMS manages keys	Hash checksums only; no AEAD tag	Database flag; server can bypass
Server-Side Encryption (SSE)	Cloud encrypts *after* receiving data	Server holds decryption keys	Vulnerable to internal tampering	Server memory dump exposes all
CipherShare (Hybrid Zero-Knowledge)	**Zero Server Plaintext** (Client RAM only)	**Client Browser Only** (Encrypted in IndexedDB)	**128-bit GMAC Tag + Canonical AAD**	**Cryptographic Envelopes + HTTP 403**

2. Cryptographic Architecture — Core Concepts

CipherShare combines symmetric authenticated encryption, asymmetric key encapsulation, canonical metadata binding, and verifiable attestations into a unified pipeline.

2.1 The Four Popa et al. Security SLA Properties

Following the formal cloud storage security definitions established by Raluca Ada Popa et al. (MIT & Microsoft Research), CipherShare guarantees four essential security properties:

1. **Confidentiality (C)**: The cloud provider or any unauthorized party cannot deduce the plaintext content of any file stored in the repository.
2. **Integrity (I)**: Every read returns the exact, unaltered payload committed by an authorized writer. Any modification is detected by the 128-bit authentication tag.
3. **Write-Serializability (W)**: Updates follow a verified sequence counter, preventing fork attacks where the cloud serves divergent views to different users.

4. **Read Freshness (F)**: Reads return the latest committed state, preventing replay of stale ciphertext using chained attestation hashes.

2.2 Step-by-Step Encryption & Sharing Protocol

1. **Local Symmetric Key Generation**: The sender's browser allocates a fresh 256-bit AES key in volatile memory and generates a 96-bit random nonce using `crypto.getRandomValues()`.
2. **Canonical AAD Assembly**: The client constructs a canonical JSON string binding file metadata: `AAD = {"fileId", "sender", "recipient", "classification", "version"}`.
3. **Payload AEAD Encryption**: The payload is encrypted via AES-256-GCM. The cipher produces the ciphertext and a 128-bit Galois Message Authentication Code (GMAC) tag over both the payload and AAD.
4. **Asymmetric Key Encapsulation (RSA-OAEP 3072)**: The client fetches the recipient's published RSA-OAEP 3072-bit public key and wraps the 256-bit AES key into an encrypted key envelope.
5. **Zero-Knowledge Cloud Dispatch**: The client uploads the ciphertext binary blob and recipient key envelope to cloud storage (Supabase S3). The server issues a Cloud Put Attestation.
6. **Isolated Download & Decryption**: When the authorized recipient requests download, the server verifies access and returns *only* their specific envelope. The recipient unwraps the AES key using their private key in browser RAM and decrypts the payload.

2.3 Worked Example — Hybrid Key Encapsulation & Envelope Generation

To illustrate the hybrid cryptographic flow, consider sharing a document (`Audit_2026.pdf` , 128 KB) from Alice to Bob:

Stage	Operation / Primitive	Mathematical / Technical Representation	Security Guarantee
1. Session Key	AES-256 Key Gen	`K_AES = Random(256 bits)` (Hex: `7f8a...3b21`)	Fresh volatile key per file
2. Nonce (IV)	96-bit Secure Vector	`IV = crypto.getRandomValues(12 bytes)`	Nonce uniqueness prevents replay
3. AAD Binding	Canonical UTF-8 JSON	`{"fileId":"f_92a","ver":"1.0","class":"CONFIDENTIAL"}`	Prevents ciphertext transplant
4. Bulk Encryption	AES-256-GCM	`Ciphertext, Tag_128 = Encrypt_GCM(K_AES, IV, Data, AAD)`	Authenticity + Confidentiality
5. Key Wrapping	RSA-OAEP 3072-bit	`Envelope = RSA_OAEP_Encrypt(PK_Bob, K_AES)` (384 bytes)	Only Bob's private key can unwrap
6. RAM Scrubbing	Memory Cleanup	Discard raw `K_AES` from volatile browser memory	Zero plaintext key persistence

3. Technical Implementation

3.1 Technology Stack

Component	Technology	Technical Purpose
Language & Frontend	TypeScript, React 18, Vite	Type-safe client presentation and execution framework
Cryptographic Engine	W3C Web Crypto API (`crypto.subtle`)	Hardware-accelerated browser-native cryptographic primitives
Client Key Vault	IndexedDB (`idb` wrapper)	Passphrase-encrypted private key storage at rest via PBKDF2
Backend REST API	Python 3.12, Flask, Flask-SQLAlchemy	Session authorization, ACL checks, and attestation tracking
Database	PostgreSQL (Supabase pooler / SQLite dev)	User metadata, public key JWKs, and tamper-evident audit logs
Cloud Object Storage	Supabase Cloud Storage (S3 API compatible)	Remote encrypted ciphertext repository with zero plaintext access
Authentication	Argon2id & PyJWT	Memory-hard password hashing and signed bearer token sessions

3.2 Cryptographic Methods Summary

Cryptographic Primitive	Implementation Details	Key Characteristics & Parameter Choices
AES-256-GCM	`crypto.subtle.encrypt("AES-GCM")`	256-bit key, 96-bit IV, 128-bit authentication tag over GF(2^128)
RSA-OAEP 3072	`crypto.subtle.wrapKey("RSA-OAEP")`	3072-bit modulus, e = 65537, SHA-256 hash, MGF1 padding
Canonical AAD	UTF-8 Canonical JSON Serialization	Contextually bound to GMAC polynomial to prevent block substitution
PBKDF2-SHA-256	`crypto.subtle.deriveKey("PBKDF2")`	600,000 iterations, 128-bit random salt, derives 256-bit AES vault key
Popa Attestation	Chained SHA-256 Hash Function	`chain_hash = SHA256(data + prev_chain_hash)` for freshness auditing
Argon2id	`passlib.hash.argon2`	Time cost = 2, memory cost = 64 MB, parallelism = 2

3.3 API Design & Endpoint Architecture

The client and server interact through a streamlined, authenticated REST protocol:

```
// 1. Client-Side Cryptographic API (W3C Web Crypto in browser volatile RAM)
static async encryptPayload(data: ArrayBuffer, key: CryptoKey, iv: Uint8Array, aad: Uint8Array):
Promise<ArrayBuffer>;
static async wrapKeyEnvelope(aesKey: CryptoKey, recipientPubKey: CryptoKey): Promise<string>;
static async unwrapKeyEnvelope(envelopeBase64: string, privateKey: CryptoKey): Promise<CryptoKey>;
static async decryptPayload(ciphertext: ArrayBuffer, key: CryptoKey, iv: Uint8Array, aad: Uint8Array):
Promise<ArrayBuffer>;
```

```
# 2. Backend Access Control & Attestation API (Flask Blueprint: /api/v1/files)
POST /api/v1/files/initiate              # Validates sender JWT, returns recipient public key JWK
POST /api/v1/files/<file_id>/upload      # Commits encrypted binary ciphertext blob to S3 storage
POST /api/v1/files/<file_id>/complete    # Stores key envelopes and issues Cloud Put Attestation
POST /api/v1/files/<file_id>/download-authorize  # ACL check; returns ONLY caller's envelope + Cloud Get
Attestation
POST /api/v1/files/verify-sla-proof      # Verifies non-repudiable Proofs of Violation (Popa Theorems 1 & 2)
```

4. Results & Performance

4.1 Key Performance Metrics

Metric	Measured Value	Operational Context
Maximum File Size	25 MB (configurable)	Client-side memory-safe streaming limit
Symmetric Encryption Throughput	**216.4 MB/s**	Hardware-accelerated AES-NI via Web Crypto API
Symmetric Encryption Latency	**4.62 ms**	1 MB payload (AES-256-GCM + 128-bit GMAC)
Symmetric Decryption Latency	**3.88 ms**	1 MB payload with AAD verification
RSA-OAEP 3072 Key Wrap Latency	**3.82 ms**	Public-key encryption of 256-bit AES key
RSA-OAEP 3072 Key Unwrap Latency	**11.45 ms**	Private-key unwrapping in recipient browser RAM
PBKDF2 Key Derivation Latency	**321.4 ms**	600,000 iterations for local IndexedDB vault unlock
Bit-Flip Tamper Detection	**100% (Instant)**	Throws `DOMException: OperationError`; 0 bytes leaked
Automated Test Pass Rate	**100% (11/11)**	6/6 Frontend Vitest (1.11s) + 5/5 Backend Pytest (3.18s)

4.2 Comparative Evaluation — Popa et al. (CloudProof) vs. CipherShare

Evaluation Dimension	Popa et al. (CloudProof, USENIX ATC '11)	CipherShare (Our Modernized Implementation)
Hash Standard	SHA-1 (Broken, vulnerable to collision attacks)	**SHA-256 / SHA-512** (Collision-resistant, FIPS 180-4)
Public Key Cryptography	1024-bit RSA (Factoring vulnerability)	**RSA-OAEP 3072-bit** (NIST compliant through 2030+)
Bulk Cipher & Mode	AES-CTR with separate RSA signatures per block	**AES-256-GCM** single-pass authenticated encryption
Transplant Resilience	None (block transplant between files undetected)	**Canonical AAD Context Binding** prevents substitution
Client Deployment	Native C# Desktop Binary (.NET / Windows Server)	**Zero-Install Web Browser** via W3C Web Crypto API
Private Key At Rest	Unprotected local OS filesystem	**PBKDF2-SHA-256 (600k iter) + AES-GCM in IndexedDB**
Encryption Throughput	~3.5 MB/s (measured in .NET runtime)	**216.4 MB/s** (in-browser hardware AES-NI execution)
Write Overhead	11.0% throughput degradation on Azure	**Negligible (< 2.5% on 1 MB payloads)**

4.3 Why Web Crypto API & IndexedDB?

The W3C Web Crypto API executes directly in browser native code (C++ in Chromium/V8), providing direct access to CPU cryptographic instructions (AES-NI and CLMUL) without JavaScript interpreter overhead. Furthermore, storing private keys in `localStorage` would leave them exposed to Cross-Site Scripting (XSS). CipherShare avoids this by deriving an ephemeral Key Encryption Key (KEK) using PBKDF2-SHA-256 and encrypting the private key at rest inside browser **IndexedDB**, guaranteeing that private keys exist unencrypted strictly in volatile RAM during active sessions.

5. Security Analysis & Threat Mitigation

Threat / Attack Vector	Attack Mechanism	Detection & Defense Mechanism	Security Outcome
Passive Eavesdropping	Intercepting ciphertext in transit or at rest	AES-256-GCM symmetric encryption (2^256 keyspace)	Complete confidentiality; data indistinguishable from random noise
Unauthorized Access (Eve)	Non-recipient requests download authorization	Server checks recipient ACL; returns HTTP 403 Forbidden	Key envelope strictly withheld; zero key material delivered
Bit-Flip Wire Tampering	Adversary flips bits in ciphertext (`byte[12] ^= 0x01`)	128-bit Galois Message Authentication Code (GMAC) verification	Decryption fails with `OperationError`; exactly 0 plaintext bytes released
Ciphertext Transplant	Adversary swaps ciphertext of File A into File B	Canonical AAD bound to GMAC polynomial tag	Metadata mismatch invalidates authentication tag immediately
Replay / Stale Data	Cloud returns old committed ciphertext version	Popa et al. Chained Attestation hash + client nonce	Attestation sequence broken; freshness violation provable
Server Compromise	Malicious admin dumps database and S3 buckets	Zero Server Plaintext Invariant: server has no keys	Stored blobs are computationally impossible to decrypt without private keys

5.1 Popa et al. Proofs of Violation in CipherShare

Under Popa et al.'s SLA enforcement theorems:

- **Proof of Authorization Violation**: When Eve attempts download, the server records an immutable audit log and returns `HTTP 403 Forbidden`. The absence of a valid recipient signature serves as non-repudiable proof that the cloud denied access to an unauthorized party.
- **Proof of Integrity Violation**: If an adversary or storage fault alters stored bytes, the client presents the cloud's signed put attestation alongside the received ciphertext. The failure of the 128-bit GMAC tag constitutes non-repudiable proof of cloud integrity failure.

---

6. Limitations & Future Scope

6.1 Current Limitations

Limitation	Technical Description	Impact & Mitigation
Browser RAM Ceiling	Web Crypto operates in-memory; files are buffered in RAM	Files > 100 MB may strain browser memory; resolved via chunked streaming
Endpoint Compromise	Client device infected with kernel-level keylogger	If the client endpoint is breached, volatile RAM can be inspected after decryption
Public Metadata Visibility	File names, sizes, and recipient emails are cataloged	Encrypted filenames and anonymous recipient IDs can be added for metadata privacy

6.2 Planned Enhancements

- **Post-Quantum Cryptography (PQC)**: Upgrade key encapsulation from RSA-OAEP 3072 to **ML-KEM (Kyber-768)** to defend against "Harvest Now, Decrypt Later" quantum attacks.
- **Hierarchical Access Control**: Implement Ciphertext-Policy Attribute-Based Encryption (CP-ABE) for role-based organizational sharing without individual recipient envelopes.
- **Zero-Knowledge Proofs (zk-SNARKs)**: Enable clients to verify storage policy compliance (e.g., file retention dates) without revealing metadata to the public directory.

---

7. Conclusion

CipherShare successfully implements and modernizes the cryptographic cloud storage guarantees formulated by **Popa et al. (CloudProof, USENIX ATC '11)**. By deploying genuine **AES-256-GCM** authenticated symmetric encryption, **Canonical AAD** contextual binding, **RSA-OAEP 3072-bit** asymmetric key encapsulation, and an **IndexedDB encrypted key vault**, the system eliminates reliance on cloud trust.

Empirical testing confirms that client-side hybrid cryptography incurs negligible performance overhead (**4.62 ms** latency, **216 MB/s** throughput) while providing robust mathematical protection: non-recipients are stopped by strict **HTTP 403 Forbidden** barriers, single-bit corruptions are stopped by 128-bit GMAC tags, and the cloud server never possesses plaintext or private keys. CipherShare demonstrates that zero-knowledge security SLAs can be seamlessly deployed in standard web browsers without requiring native plugins or compromising cloud scalability.

Project Repository: `https://github.com/ciphershare/ciphershare`
Group Number: Group - 22
Evaluation Scope: Academic Applied Cryptography & Information Security Project